ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

- Write a default constructor, a parameterised constructor and overloaded constructors for the three situations
- Use a member initializer list in every constructor
- Use constructor delegation to avoid repeating code
- Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

- Explain when the compiler supplies a default constructor and when it does not
- Explain why member initializer lists are preferred to assignment inside the constructor body
- Explain the order in which members are initialised (declaration order, not list order)
- Draw the state of three objects after construction

Experimental Investigation

You must:

- Create Book objects with every constructor and print them
- Remove the default constructor and record the compiler error when an array of Books is created
- Demonstrate a member initialised in the wrong order and the -Wreorder warning
- Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

- Implement the class first without a copy constructor and observe the problem
- Implement a deep-copying copy constructor, a copy assignment operator and a destructor
- Verify that copies are independent
- Run the program under -fsanitize=address (or valgrind) for both versions

Design & Analysis

You must:

- Draw memory diagrams for a shallow copy and a deep copy
- Explain why the shallow version crashes at the end (double delete)
- State the Rule of Three and explain why the three functions go together
- Explain the three situations in which the copy constructor is called

Experimental Investigation

You must:

- Add a message in the copy constructor and count how many copies are made when a Prescription is passed by value and by const reference
- Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines
- Report the sanitizer output for both versions
- Test self-assignment (p = p)

Conclusion

When can you rely on the compiler-generated copy constructor, and when must you write your own?

Complete the Code

Complete TODO 1 to TODO 3 below so that every copy is independent, and execute the program under -fsanitize=address.

```
#include <iostream>

#include <string>


class Prescription {

    int n;

    std::string* medicines;          // dynamically allocated array


public:

    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
```

```
    // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every
    //   medicine (deep copy)



    // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,
    //   release the old array and return *this



    ~Prescription()
    {
        // TODO 3: Release the dynamically allocated array


    }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};

int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;              // copy constructor
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;                           // copy assignment
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
```

```
        << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Delete your copy constructor, run the program again under the sanitizer and record the error reported.

Alternative Concept and Complexity Analysis

If the medicine names were stored in a std::vector<std::string> instead of a dynamically allocated array, the compiler-generated copy constructor would already make a deep copy (the Rule of Zero). What would the solution be? Analyse the time O(n) and space O(n) to copy a prescription of n medicines in both designs, and compare them with a shallow copy (O(1)).

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

- Write a dynamic constructor that allocates a two-dimensional seat map with new
- Write a destructor that releases every allocation and prints a summary
- Create events on the stack, on the heap (new/delete) and inside a block
- Book and cancel seats

Design & Analysis

You must:

- Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects

- Explain what happens if delete is forgotten, and if delete is used instead of delete[]
- Explain why the destructor takes no arguments and cannot be overloaded
- Draw the memory layout of the seat map

Experimental Investigation

You must:

- Print messages in the constructor and destructor and record the order for each kind of object
- Run with -fsanitize=address, first with a missing delete and then with the correct code
- Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps
- Test an event with zero rows

Conclusion

How does tying memory release to the destructor (the RAII idea) make the program safer?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
#include <string>

struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

int main()
```

```
{
   Logger a("A");
   {
      Logger b("B");
   }
   use(a);
   Logger* p = new Logger("P");
   delete p;
   std::cout << "end ";
}
```

My predicted output (write before running):

Explain when each object was created and destroyed, and why the copy was made and destroyed around use().

Alternative Concept and Complexity Analysis

If the seat map were one block of rows × seats elements (index r × seats + s) or a std::vector<std::vector<char>>, instead of an array of row pointers each allocated with new, what would the solution be? Analyse the access time (O(1) in all), the number of allocations (1 versus rows + 1) and the extra space for the row pointers.

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

Implementation Tasks

You must:

- Write a Money class storing paise in a long long
- Overload +, -, +=, == and < as appropriate
- Overload << (and >>) for printing and reading amounts
- Write the bill calculation using the overloaded operators

Design & Analysis

You must:

- Explain why + returns a new object while += returns a reference to *this
- Decide which operators are members and which are friends, and justify each
- Explain why << must be a non-member function
- Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

- Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50)
- Test subtraction that produces a negative amount and decide how to handle it
- Compare the readability of the bill code with and without operator overloading
- Compare the result with a double-based version for 1,000,000 additions

Engineering Decision

Should Money allow multiplication by another Money? Decide which operators make sense for money and which do not.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

#include <iomanip>

#include <iostream>

```
class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: Overload + as a member function that returns a new Money object



    // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"

};

int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
}
```

Expected output:

Rs 200.00

true

Add a test where paise carry into rupees (Rs 0.75 + Rs 0.50).

Alternative Concept and Complexity Analysis

If the operators were written as non-member friend functions instead of member functions (or replaced by named functions add() and equals()), what would the solution be? Analyse whether time or space changes (all are O(1)), and explain which form allows 100 + m as well as m + 100.

Q5 – Overloading Increment, Comparison and Subscript Operators 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

- Overload prefix and postfix ++ for Time
- Overload == and < for Time
- Overload [] for Playlist with a range check, in both a const and a non-const version
- Use the operators in a short playback simulation

Design & Analysis

You must:

- Explain how the compiler tells prefix from postfix ++ (the int dummy parameter)
- Explain why postfix ++ must return a copy and is slower
- Explain why operator[] needs a const and a non-const version
- Explain what should happen when an index is out of range

Experimental Investigation

You must:

- Verify the values of a and b after a = t++ and b = ++t
- Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2
- Test playlist[-1] and playlist[size]
- Test minute and hour roll-over (59:59 → 1:00:00)

Conclusion

Why do many programmers prefer ++i to i++ for class types?

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; }                    // prefix
    Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix
};


int main()
{
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

My predicted output (write before running):

Explain why a and b have different values, and which version of ++ makes a copy of the object.

Alternative Concept and Complexity Analysis

If playlist access used an at(i) function that throws std::out_of_range, or an unchecked operator[] like a built-in array, instead of operator[] with a range check that prints an error,

what would the solution be? Analyse the time per access (O(1) in all, with one extra comparison for checked access) and measure the cost of the check for 10,000,000 accesses.

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

Implementation Tasks

You must:

- Implement Text with a constructor, copy constructor, copy assignment and destructor
- Overload + for concatenation, == and != for comparison and << for printing
- Prevent self-assignment problems
- Test long messages and empty messages

Design & Analysis

You must:

- List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why
- Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type
- Explain why operator= must be a member function
- Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

- Count the copies made when a + b + c is evaluated
- Run the program under the address sanitizer and fix any reported error
- Compare your Text class with std::string for 100,000 concatenations
- Test assigning a Text to itself

Conclusion

Your Text class works, but std::string exists. What did writing Text teach you, and when would you still write such a class?

Alternative Concept and Complexity Analysis

If the Text class were replaced by std::string, or if + were implemented with += on a copy (Text operator+(Text a, const Text& b) { a += b; return a; }), what would the solution be? Analyse the time to evaluate a + b + c for strings of lengths n1, n2, n3 (the characters copied) and the temporary space created in each version.

Q7 – Design a Complex-Number Class from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the data and the operations needed
- Identify which constructors are needed (default, from real part, from real and imaginary parts)
- State the requirements as measurable statements

Stage 2 – Design | 3 Marks

- Design the Complex class with constructors and overloaded +, -, *, /, == and <<
- Decide which operators are members and which are friends
- Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

- Implement the class and compute the impedance of a series and a parallel RLC circuit

- Verify the results by hand calculation
- Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

- Compare your class with std::complex<double>
- Explain why each constructor and operator is needed
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C++ source code for all questions
- Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ASSIGNMENT 3 (CO3) - MODEL ANSWERS

Constructors, Destructors and Operator Overloading in C++

Name: _____ Register No.: _____ GitHub: github.com/<you>/DSA01-<RegNo>-Assignment3

Compiler used for every output shown here: g++ 13.3.0 (Ubuntu 24.04). **Command:** `g++ -std=c++17 -Wall -Wextra -fsanitize=address -g file.cpp -o file`

Important: all programs below were compiled and run, and the outputs/sanitizer messages/timings shown are real. Timings depend on your machine, so re-run them yourself and report your own numbers. The assignment also requires you to commit each task separately to GitHub, take your own screenshots, and write the *Predict the Output* answers before running. Use this document as a reference to check your work and to prepare for the viva, and write the explanations in your own words.

Q1 - Constructors and Constructor Overloading (6)

Implementation (Task01/q1.cpp)

```
#include <iostream>
#include <string>
class Book {
    std::string title;
    std::string author;
    int copies;
    static int valid(int c) { return c < 0 ? 0 : c; }   // single validation point
public:
    // full-detail constructor (all others delegate to it)
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(valid(c)) {}
    Book(const std::string& t) : Book(t, "Unknown", 1) {}   // donated book
    Book() : Book("Untitled", "Unknown", 0) {}              // empty record
    friend std::ostream& operator<<(std::ostream& o, const Book& b) {
        return o << '[' << b.title << " | " << b.author << " | copies=" << b.copies << ']';
    }
};
int main() {
    Book a("DSA in C++", "Sahni", 5), b("Donated Novel"), c, d("Bad", "X", -7);
    std::cout << a << '\n' << b << '\n' << c << '\n' << d << '\n';
}
```

```
[DSA in C++ | Sahni | copies=5]
[Donated Novel | Unknown | copies=1]
[Untitled | Unknown | copies=0]
[Bad | X | copies=0]
```

Design: One 3-argument constructor does the real work; the title-only (donated) and default (empty record) constructors *delegate* to it, so the check `valid(c)` exists in exactly one place. No Book can hold a negative count (Book("Bad","X",-7) is stored as 0). All constructors use initializer lists.

Analysis

- **Default constructor:** the compiler supplies one only if the class declares *no* constructor at all. As soon as any constructor is written (even a parameterised one) the implicit default one disappears.
- **Initializer list vs assignment:** members are initialised before the body runs. In the body, a member is first default-constructed and then assigned (two steps; for std::string this wastes work). const members, reference members and members without a default constructor can only be set in the list.
- **Order:** members are initialised in the order of *declaration* in the class, not the order written in the list. Writing them in a different order triggers -Wreorder and can read uninitialised values.
- **State of three objects:** a = {title:"DSA in C++", author:"Sahni", copies:5}; b = {"Donated Novel", "Unknown", 1}; c = {"Untitled", "Unknown", 0}.

Experiments (real compiler messages)

```
Remove default ctor, then: Book arr[3];
q1_noarr.cpp:3:24: error: no matching function for call to 'Book::Book()'

struct P { int a; int b; P(int x) : b(x), a(b + 1) {} };
warning: 'P::b' will be initialized after [-Wreorder]
         'int P::a' [-Wreorder]      (a is built first, from uninitialised b)

Negative copies: Book("Bad","X",-7) -> copies=0 (clamped by valid())
```

Design question - explicit

Mark a single-argument constructor `explicit` when an implicit conversion would be surprising or unsafe (e.g. a size or count silently turning into an object).

```
struct Box { /*explicit*/ Box(int n) : v(n) {} int v; };
void show(Box b);
show(42);   // compiles without explicit
// with explicit:  error: could not convert '42' from 'int' to 'Box'
// must write  show(Box(42));
```

Alternative: one constructor with default arguments

```
Book(std::string t = "", std::string a = "Unknown", int c = 0)
    : title(t), author(a), copies(c < 0 ? 0 : c) {}
```

Creation time and space are O(1) in both designs. The default-argument version has only one constructor, so the copies check is written **once** and needs no delegation. The overloaded+delegation design also checks it once (in the target constructor) but needs three declarations, and lets you give different default meanings (e.g. donated = 1 copy). Default arguments are shorter; overloads are more flexible.

Q2 - Copy Constructors: Shallow and Deep Copies (7)

Complete the Code (all TODOs filled) - q2.cpp

```
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
    // TODO 1: deep-copy constructor
    Prescription(const Prescription& o) : n(o.n), medicines(new std::string[o.n]) {
        for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i];
    }
    // TODO 2: copy assignment
    Prescription& operator=(const Prescription& o) {
        if (this != &o) {
            std::string* tmp = new std::string[o.n];
            for (int i = 0; i < o.n; ++i) tmp[i] = o.medicines[i];
            delete[] medicines;
            medicines = tmp;
            n = o.n;
        }
        return *this;
    }
    ~Prescription() {
        // TODO 3
        delete[] medicines;
    }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1) << " | " << r.get(0) << ' ' << r.g
    p = p; // self-assignment test
    std::cout << p.get(0) << '\n';
}
```

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
Ibuprofen        <- p = p (self-assignment) is safe
```

Shallow version (no copy ctor/assignment) under -fsanitize=address

```
==178==ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090
READ of size 8 at 0x507000000090 thread T0
    #0 ... in Prescription::~Prescription() q2_shallow.cpp:10
    #1 ... in main q2_shallow.cpp:26
0x507000000090 is located 0 bytes inside of 72-byte region
```

```
freed by thread T0 here:  operator delete[] ... ~Prescription()
```

Also, in the shallow version q = p copies only the pointer, so q.set(1,"Cetirizine") changes p as well (the original changes!). The fixed (deep copy) version gives the expected output above and a clean sanitizer run.

Memory diagrams

```
SHALLOW COPY (compiler default)            DEEP COPY (our copy ctor)
 p: [n=2][medicines]--+                     p: [n=2][medicines]--> [Paracetamol][ORS]
 q: [n=2][medicines]--+--> [Para][ORS]      q: [n=2][medicines]--> [Paracetamol][ORS]  (new array)
 one array, two owners -> double delete[]   two arrays, one owner each
```

Analysis

- **Crash:** at end of main, q's destructor frees the array, then p's destructor calls delete[] on the same (already freed) address: double delete / use-after-free.
- **Rule of Three:** if a class needs a user-defined destructor, copy constructor or copy assignment (because it owns a resource), it almost certainly needs all three, since each of them must agree on who owns and who frees the resource.
- **Copy constructor is called when:** (1) an object is initialised from another (`Prescription q = p;`), (2) an object is passed by value to a function, (3) an object is returned by value (when not elided by RVO/copy elision).

Measurements (g++ -O2, my machine - repeat on yours)

Medicines	Time per deep copy
10	0.32 microseconds
1,000	34 microseconds
100,000	4.0 milliseconds

Time grows linearly, O(n). Passing by value prints one 'copy' per call; by const reference prints none:

```
by value: copy dtor
by const ref: (no copy)
```

Conclusion and Alternative

Rely on the compiler-generated copy when all members are values or self-copying types (int, std::string, std::vector, smart pointers with the right semantics): the Rule of Zero. Write your own when the class owns a raw resource (new[], file handle, socket).

With std::vector<std::string>: no destructor/copy ctor/assignment is needed; the default copy is already deep. Copying is O(n) time and O(n) space in both designs; a shallow copy is O(1) but unsafe.

```
class Prescription { std::vector<std::string> medicines; public: explicit Prescription(int c): medicines(c) {} ... };
```

Q3 - Dynamic Constructors and Destructors (7)

Implementation - q3_event.cpp (2-D seat map, rows of char)

```
#include <iostream>
class Event {
    int rows, seats, booked;
    char** map;
public:
    Event(int r, int s) : rows(r), seats(s), booked(0), map(nullptr) {
        map = new char*[rows];
        for (int i = 0; i < rows; ++i) { map[i] = new char[seats]; for (int j = 0; j < seats; ++j) map[i][j] = 'O'; }
        std::cout << "[ctor " << rows << "x" << seats << "]\n";
    }
    Event(const Event&) = delete;
    Event& operator=(const Event&) = delete;
    bool book(int r, int s)   { if (r<0||r>=rows||s<0||s>=seats||map[r][s]=='X') return false; map[r][s]='X'; ++booked;
    bool cancel(int r, int s) { if (r<0||r>=rows||s<0||s>=seats||map[r][s]=='O') return false; map[r][s]='O'; --booked;
    ~Event() {
        std::cout << "[dtor] booked=" << booked << " free=" << rows*seats-booked << "\n";
        for (int i = 0; i < rows; ++i) delete[] map[i];
        delete[] map;
    }
};
int main() {
    Event e1(2, 3);               // stack
    e1.book(0,1); e1.book(1,2); e1.cancel(0,1);
    { Event e2(1, 2); e2.book(0,0); }   // block scope
```

```
    Event* e3 = new Event(3, 3);  // heap
    e3->book(2,2);
    delete e3;
    Event e0(0, 5);               // zero rows
}
```

```
[ctor 2x3]
[ctor 1x2]
[dtor] booked=1 free=1      <- block-scope e2 destroyed at '}'
[ctor 3x3]
[dtor] booked=1 free=8      <- delete e3
[ctor 0x5]
[dtor] booked=0 free=0      <- e0 (zero rows works: new char*[0] is legal)
[dtor] booked=1 free=5      <- e1 (stack objects die in reverse order: e0 first, then e1)
```

Analysis

- **Order:** constructors run at the point of definition; stack/block objects are destroyed in *reverse* order of creation when the scope ends; a heap object is destroyed only when delete is called.
- **Forgotten delete:** the destructor never runs and the memory leaks (LeakSanitizer reports 'Direct leak', shown below). **delete instead of delete[]:** undefined behaviour (for arrays only the first element is destroyed / the allocator may be corrupted; ASan reports new[]-delete mismatch).
- **Destructor has no arguments and cannot be overloaded:** it is called implicitly by the system at end of lifetime, so there is no place to supply arguments, and there must be exactly one cleanup routine per class.
- **Layout:** `map` -> [ptr0][ptr1]...[ptr(rows-1)] ; each ptr_i -> [seat0][seat1]...[seat(seats-1)] (rows+1 separate heap blocks).

```
Leak test (q3_leak.cpp, delete forgotten):
==185==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 8 byte(s) in 1 object(s) allocated from: ... main q3_leak.cpp:2
```

Memory for seat maps (char seats, 8-byte row pointers, excl. allocator overhead)

Size	Formula rows*8 + rows*seats	Bytes
10 x 20	80 + 200	280
100 x 200	800 + 20,000	20,800
1,000 x 2,000	8,000 + 2,000,000	2,008,000 (about 1.9 MiB)

Conclusion (RAII)

Acquiring the memory in the constructor and releasing it in the destructor means cleanup happens automatically on every exit path (return, exception, end of block), so leaks and forgotten frees are eliminated.

Predict the Output

```
#include <iostream>
#include <string>
struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};
void use(Logger) { std::cout << "use "; }
int main(){
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";
}
```

```
+A +B -B copyAc use -Ac +P -P end -A
```

- **+A**: a created. **+B -B**: b created and destroyed at the end of the inner block.
- **copyAc use -Ac**: use(a) takes Logger by value, so the copy constructor builds a temporary named 'Ac' (A + "c"); the function prints 'use'; the copy is destroyed when the function returns (parameter lifetime).
- **+P -P**: heap object created, then delete runs its destructor. **end** printed, then main finishes and **-A** destroys a.

Alternative: one block or vector of vectors

A single block `char* seats = new char[rows*cols]` accessed as seats[r*cols + s]: 1 allocation, no row-pointer space, contiguous memory (better cache use). std::vector<std::vector<char>>: rows+1 allocations, extra space for the vector headers, but no manual delete. Access is O(1) in all; allocations: 1 vs rows+1; extra space: none vs rows pointers (8*rows bytes).

Q4 - Overloading Arithmetic and Stream Operators (7)

Complete the Code (TODO 1 and 2) - q4.cpp

```
#include <iomanip>
#include <iostream>
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    // TODO 1
    Money operator+(const Money& o) const { Money r; r.paise = paise + o.paise; return r; }
    Money operator-(const Money& o) const { Money r; r.paise = paise - o.paise; return r; }
    Money& operator+=(const Money& o) { paise += o.paise; return *this; }
    // TODO 2
    bool operator==(const Money& o) const { return paise == o.paise; }
    bool operator<(const Money& o) const { return paise < o.paise; }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long v = m.paise < 0 ? -m.paise : m.paise;
        return os << (m.paise < 0 ? "-" : "") << "Rs " << v / 100 << '.'
                  << std::setw(2) << std::setfill('0') << v % 100 << std::setfill(' ');
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    std::cout << Money(0,75) + Money(0,50) << '\n';        // carry test
    std::cout << Money(10) - Money(25, 50) << '\n';        // negative test
    std::cout << (Money(5) < Money(6)) << '\n';
}
```

```
Rs 200.00
true
Rs 1.25          <- 0.75 + 0.50: paise carry into rupees
-Rs 15.50        <- negative result printed with a sign
true
```

Analysis

- **+ vs +=:** a + b creates a new object and leaves operands unchanged (returns by value); += modifies the left operand and returns `*this` by reference to allow chaining (a += b += c).
- **Members:** +, -, +=, ==, < use the left operand's data, so members. **Friend:** operator<< (and >>).
- **Why << is a non-member:** the left operand is std::ostream, which we cannot modify, so it cannot be a member of Money; it is a friend so it can read private paise.
- **Translation:** a + b becomes a.operator+(b) (member) or operator+(a, b) (non-member).
- **Negative amounts:** Money keeps a signed long long, so the result is stored as a negative number and printed with a '-' sign. Alternatively throw an exception for 'insufficient balance' if business rules forbid negatives.
- **double comparison:** adding 0.10 one million times in double gives about 100000.00000133 (not exactly 100000), while Money (integer paise) is exact: 100000.00. Money is therefore the right choice for billing.

Engineering decision

Money x Money makes no sense (Rs x Rs = Rs squared). Sensible: Money + Money, Money - Money, Money * number, Money / number, comparisons, unary minus. Not sensible: Money * Money, Money + plain number (unclear units).

Alternative: non-member friends / named functions

As friends: `friend Money operator+(const Money& a, const Money& b);`. Time and space remain O(1). Because the left operand of a non-member is also converted implicitly, **both 100 + m and m + 100** work (via Money(long long)); a member operator+ only supports m + 100. Named add()/equals() also give O(1) but make bill expressions hard to read.

Q5 - Increment, Comparison and Subscript Operators (6)

Time and Playlist - q5_time.cpp

```
#include <iostream>
#include <string>
#include <stdexcept>
class Time {
    int h, m, s;
public:
    Time(int hh=0,int mm=0,int ss=0):h(hh),m(mm),s(ss){}
    Time& operator++(){ if(++s==60){s=0; if(++m==60){m=0; ++h;}} return *this; }
    Time operator++(int){ Time old=*this; ++*this; return old; }
    bool operator==(const Time&o)const{return h==o.h&&m==o.m&&s==o.s;}
    bool operator<(const Time&o)const{ return h!=o.h? h<o.h : m!=o.m? m<o.m : s<o.s; }
    friend std::ostream& operator<<(std::ostream&os,const Time&t){
        return os<<t.h<<':'<<(t.m<10?"0":"")<<t.m<<':'<<(t.s<10?"0":"")<<t.s; }
};
class Playlist {
    std::string tracks[3] = {"Song A","Song B","Song C"};
    int n = 3;
public:
    std::string& operator[](int i){ if(i<0||i>=n) throw std::out_of_range("index"); return tracks[i]; }
    const std::string& operator[](int i) const { if(i<0||i>=n) throw std::out_of_range("index"); return tracks[i]; }
};
int main(){
    Time t(0,59,59);
    Time a = t++;  Time b = ++t;
    std::cout << a << " " << b << " " << t << "\n";
    Playlist p;
    std::cout << p[0] << "\n";
    try { p[-1]; } catch (const std::out_of_range&) { std::cout << "p[-1] rejected\n"; }
    try { p[3];  } catch (const std::out_of_range&) { std::cout << "p[size] rejected\n"; }
}
```

```
0:59:59 1:00:01 1:00:01
Song A
p[-1] rejected
p[size] rejected
```

a = t++ gets the old value 0:59:59; b = ++t gets the new value; 59:59 rolls over to 1:00:00 (minutes and seconds reset, hour increments).

Analysis

- **Prefix vs postfix:** the compiler calls operator++(int) for t++ (the unused int is only a tag) and operator++() for ++t.
- **Postfix returns a copy** of the old value, so it creates a temporary and is slower for large classes.
- **const and non-const operator[]:** a const Playlist can only call the const version (returns const reference, read-only); the non-const version allows modification, e.g. playlist[2] = "X".
- **Out of range:** throw std::out_of_range (or print an error and stop); never read memory outside the array.

Timing (100,000,000 increments, my machine)

Flag	Prefix	Postfix
-O0	114 ms	183 ms
-O2	about 0 ms (loop optimised away)	about 0 ms

At -O0 postfix is slower because of the extra copy; at -O2 the compiler removes both loops for this simple type, so use a larger class or a volatile/escape sink to measure fairly.

Conclusion

++i avoids creating a temporary old value, so for class types (iterators, big objects) it is never slower than i++ and often faster.

Predict the Output

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; }
    Counter operator++(int) { Counter old = *this; ++v; return old; }
};
```

```
int main() {
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

```
5 7 7
```

c++ returns the old copy (5) and sets c=6; ++c sets c=7 and returns *this, so b = 7; final c = 7. Only the **postfix** version makes a copy of the object.

Alternative: at(i) or unchecked operator[]

at(i) throws std::out_of_range, the caller can catch it. Unchecked []: fastest, but undefined behaviour out of range. All are O(1) per access; the checked version adds one comparison (about one extra instruction), which is negligible; measure with 10,000,000 accesses.

Q6 - The Rules of Operator Overloading (7)

Text class - q6.cpp

```
#include <cstring>
#include <iostream>
class Text {
    size_t len; char* s;
public:
    Text(const char* c = "") : len(std::strlen(c)), s(new char[std::strlen(c)+1]) { std::strcpy(s, c); }
    Text(const Text& o) : len(o.len), s(new char[o.len+1]) { std::strcpy(s, o.s); }
    Text& operator=(const Text& o) {
        if (this != &o) { char* t = new char[o.len+1]; std::strcpy(t, o.s); delete[] s; s = t; len = o.len; }
        return *this;
    }
    ~Text() { delete[] s; }
    Text operator+(const Text& o) const {
        Text r; delete[] r.s; r.len = len + o.len; r.s = new char[r.len+1];
        std::strcpy(r.s, s); std::strcat(r.s, o.s); return r;
    }
    bool operator==(const Text& o) const { return std::strcmp(s, o.s) == 0; }
    bool operator!=(const Text& o) const { return !(*this == o); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.s; }
};
int main() {
    Text a("Hello, "), b("SMS "), c("World"), e;
    Text d = a + b + c;
    std::cout << d << '\n' << (d == Text("Hello, SMS World")) << (d != e) << '\n';
    d = d; std::cout << d << '\n';
    std::cout << "[" << e + e << "]\n";
}
```

```
Hello, SMS World
11           <- (d==Text(...)) true, (d!=e) true
Hello, SMS World   <- d = d safe
[]             <- empty + empty works
```

Analysis

- **Cannot be overloaded:** :: (scope; operates on names, not objects), . (member access), .* (pointer-to-member access), ?: (ternary; cannot defer evaluation of operands), sizeof (compile-time, must have fixed meaning).
- **Rules:** precedence, associativity and arity (number of operands) are fixed; new operators cannot be invented; at least one operand must be a user-defined type (you cannot redefine int + int).
- **operator= must be a member** so the compiler always knows the left operand is a modifiable object of the class (a non-member could be shadowed or implicitly converted).
- **Friend:** use when a non-member needs private data, e.g. operator<< or a symmetric binary operator with conversions on the left.
- **Copies in a + b + c:** (a + b) creates one temporary, then that temporary + c creates a second one: 2 new objects (copy elision may remove the return copy).
- **Self-assignment:** guarded by `if (this != &o)`; without it, delete[] s would free the data before copying from it. Run under ASan: no errors for the code above.

Conclusion

Writing Text teaches ownership, Rule of Three, deep copy and exception-safe assignment. You would still write such a class when you must control memory exactly (embedded systems, custom allocators, interfacing C APIs) or to learn.

Alternative: std::string / += based +

`Text operator+(Text a, const Text& b) { a += b; return a; }`: the left operand is copied once (by value) and then appended. For a+b+c, characters copied: version 1: (n1+n2) + (n1+n2+n3) = 2n1 + 2n2 + n3; version 2 with a moved/appended temporary: about n1 + n2 + n3 with std::string (it appends in place, amortised). Temporary space: one extra string of n1+n2 in both, but std::string avoids the extra copy and also uses small-string optimisation.

Q7 - Complex-Number Class for AC Circuit Impedance (10)

Stage 1 - Interpret (2)

- **Data:** real part (resistance) and imaginary part (reactance), both double. **Operations:** +, -, *, / (series and parallel combination), ==, magnitude, printing.
- **Constructors:** default Complex() = 0+0j; Complex(double r) = r + 0j (a resistor); Complex(double r, double i) (inductor/capacitor).
- **Measurable requirements:** (R1) (3+4j)+(1-2j) = 4+2j; (R2) series Z = R + jwL - j/(wC) exact to 1e-9; (R3) |3+4j| = 5; (R4) parallel (Z1*Z2)/(Z1+Z2) correct; (R5) dividing by 0+0j never crashes: it reports an error.

Stage 2 - Design (3)

Operator	Formula	Form
+	(a+c) + (b+d)j	member
-	(a-c) + (b-d)j	member
*	(ac - bd) + (ad + bc)j	member
/	((ac + bd) + (bc - ad)j) / (c^2 + d^2); throw if c^2+d^2 = 0	member
==	|re1-re2| < eps and |im1-im2| < eps (floating-point tolerance)	member
<<	prints a + jb or a - jb	friend (left operand is ostream)
double + Complex	Complex(l) + r	friend (so 5 + z works)

Stage 3 - Implement and Verify (3) - q7.cpp

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double r) : re(r), im(0) {}
    Complex(double r, double i) : re(r), im(i) {}
    double magnitude() const { return std::sqrt(re*re + im*im); }
    Complex operator+(const Complex& o) const { return Complex(re+o.re, im+o.im); }
    Complex operator-(const Complex& o) const { return Complex(re-o.re, im-o.im); }
    Complex operator*(const Complex& o) const { return Complex(re*o.re - im*o.im, re*o.im + im*o.re); }
    Complex operator/(const Complex& o) const {
        double d = o.re*o.re + o.im*o.im;
        if (d == 0.0) throw std::domain_error("division by zero impedance");
        return Complex((re*o.re + im*o.im)/d, (im*o.re - re*o.im)/d);
    }
    bool operator==(const Complex& o) const { return std::fabs(re-o.re)<1e-9 && std::fabs(im-o.im)<1e-9; }
    friend Complex operator+(double l, const Complex& r) { return Complex(l) + r; }
    friend std::ostream& operator<<(std::ostream& os, const Complex& c) {
        return os << c.re << (c.im < 0 ? " - j" : " + j") << std::fabs(c.im);
    }
};
int main() {
    const double w = 1000, R = 10, L = 0.02, C = 50e-6;   // R=10 ohm, L=20 mH, C=50 uF, w=1000 rad/s
    Complex ZR(R), ZL(0, w*L), ZC(0, -1/(w*C));            // 10, j20, -j20
    Complex series = ZR + ZL + ZC;
```

```
    std::cout << "Series   Z = " << series << "  |Z| = " << series.magnitude() << "\n";
    Complex par = (ZR * ZL) / (ZR + ZL);                    // R || L
    std::cout << "R||L     Z = " << par << "  |Z| = " << par.magnitude() << "\n";
    Complex Z3 = ZR / (Complex(1) + ZR/ZL + ZR/ZC);       // 1/(1/R+1/ZL+1/ZC) via R/(1+R/ZL+R/ZC)
    std::cout << "R||L||C  Z = " << Z3 << "  |Z| = " << Z3.magnitude() << "\n";
    try { std::cout << ZR / Complex(0, 0) << "\n"; } catch (const std::exception& e) { std::cout << "Error: " << e.what
}
```

```
Series   Z = 10 + j0  |Z| = 10
R||L     Z = 8 + j4  |Z| = 8.94427
R||L||C  Z = 10 + j0  |Z| = 10
Error: division by zero impedance
```

Hand check (R=10, L=20 mH, C=50 uF, w=1000): XL = wL = 20, XC = 1/(wC) = 20, so series Z = 10 + j(20-20) = 10 + j0, |Z| = 10 (resonance). R||L: (10 x j20)/(10 + j20) = j200(10 - j20)/(100+400) = (4000 + j2000)/500 = 8 + j4, |Z| = sqrt(80) = 8.944. Dividing by 0 + 0j is detected (c^2+d^2 = 0) and an exception is thrown and caught, so no crash or NaN.

Stage 4 - Defend (2)

- **vs std::complex<double>:** std::complex is tested, optimised, has sqrt, polar, abs, etc. Ours is educational, allows a custom error policy (exception on /0), and shows operator design.
- **Why each constructor:** default (arrays/containers), from real (resistors, implicit 5 + z), from real+imag (inductors/capacitors). **Each operator:** + series, / and * parallel formula, == for tests, << for output.
- **Limitation:** double rounding errors (comparison needs a tolerance), no polar form, no +=, -=, no unary minus, no overflow check for very large values.

Alternative: std::complex<double>

Replace Complex with std::complex<double>; operators come for free: `auto Z = (Z1*Z2)/(Z1+Z2);` `std::abs(Z);`. Time per operation O(1), space 2 doubles in both. Accuracy: std::complex division is typically more careful about overflow/underflow; but dividing by zero yields inf/NaN silently, so you must add your own check (`if (Z2 == 0.0) ...`) while our class has the check built in.

Submission checklist

- Repository name: DSA01-<RegNo>-Assignment3 with folders Task01 ... Task07; one commit per task.
- README.md: name, register number, g++ version, exact compile command, run steps, one-line result per task.
- Take screenshots of each output and each ASan run; write your predictions for Q3 and Q5 before running them.
- Re-measure all timings on your machine and quote those numbers.